Semantic tagging

Haystack v2 — Current Implementation · built Nov 13, 2026

Haystack v2 — Current Implementation

1. Database Schema

1.1 HAYSTACK_TAGS — Tag Dictionary

SQL

```sql
-- Migration: m20260607_add_haystack_v2.rs
CREATE TABLE IF NOT EXISTS HAYSTACK_TAGS (
    tag_name   TEXT PRIMARY KEY,
    doc        TEXT,
    category   TEXT NOT NULL DEFAULT 'custom',  -- 'haystack' | 'custom'
    deprecated INTEGER NOT NULL DEFAULT 0,
    source     TEXT DEFAULT 'user'              -- 'user' | 'v4'
);
```

Populated by:

- `POST /api/haystack/sync` — fetches standard tags from `https://project-haystack.org/download/defs.json`, parses `rows[].def.val` → `tag_name`, `rows[].doc` → `doc`, `rows[].is` → parent relations
- Frontend Standard Tags page "Sync" button calls this endpoint
- Custom tags created via `POST /api/haystack/tags` or the Custom Tags page

1.2 HAYSTACK_TAG_RELATIONS — Tag Inheritance

SQL

```sql
CREATE TABLE IF NOT EXISTS HAYSTACK_TAG_RELATIONS (
    tag_name   TEXT NOT NULL,
    parent_tag TEXT NOT NULL,
    PRIMARY KEY (tag_name, parent_tag)
);
```

Populated during sync. Example: `temp` → parent `point`, `outsideAirTemp` → parent `air`, `temp`, `point` (multi-parent).

1.3 HAYSTACK_POINT_TAGS — Point ↔ Tag Mapping

```SQL
CREATE TABLE IF NOT EXISTS HAYSTACK_POINT_TAGS (
    serial_number INTEGER NOT NULL,
    point_type    TEXT NOT NULL,    -- 'INPUT' | 'OUTPUT' | 'VARIABLE'
    point_index   TEXT NOT NULL,    -- 1-based index as string
    point_id      TEXT NOT NULL,    -- 'dev237219.in5'
    tag_name      TEXT NOT NULL,
    PRIMARY KEY (serial_number, point_type, point_index, tag_name)
);

CREATE INDEX IF NOT EXISTS idx_hpt_serial ON HAYSTACK_POINT_TAGS (serial_number);
CREATE INDEX IF NOT EXISTS idx_hpt_tag ON HAYSTACK_POINT_TAGS (tag_name);
```

One row per tag per point. Multiple tags = multiple rows. Primary key prevents duplicates.

---

2. Auto-Tagging Engine

File: `api/src/t3_device/haystack_tags_service.rs` **Function:** `auto_tag_point()`

2.1 Triggers

Trigger	When
FFI sync (`t3_ffi_sync_service.rs`)	Every INPUTS/OUTPUTS/VARIABLES row inserted or updated
`POST /api/haystack/rebuild`	Manual trigger — clears all tags for given serials, re-tags all points

2.2 Logic

TEXT

```
auto_tag_point(db, point_table, serial_number, point_index, label, digital_analog, units
  │
  ├─ 1. Normalize point_table → point_type (INPUTS→INPUT, OUTPUTS→OUTPUT, VARIABLES→VARI
  ├─ 2. Build point_id: "dev{serial}.{in|out|var}{index}"
  ├─ 3. Check if point already has tags → skip (don't override manual assignments)
  ├─ 4. Base tag: "point" (all points)
  ├─ 5. Type-based tags:
  │     INPUT     → add "sensor"
  │     OUTPUT    → digital_analog==1 → add "cmd"
  │               → digital_analog!=1 → add "actuator"
  ├─ 6. Units-based tags (lowercase substring match):
  │     deg.c / celsius       → "temp", "degC"
  │     deg.f / fahrenheit    → "temp", "degF"
  │     %rh / humidity        → "humidity"
  │     ppm                   → "co2"
  │     pa / pascal           → "pressure"
  │     cfm                   → "air", "flow"
  │     kw                    → "power"
  │     volt                  → "voltage"
  │     amp / ma              → "current"
  │     % (standalone)        → "percent"
  │     fpm                   → "air", "velocity"
  └─ 7. Label-based tags (lowercase substring match):
        temp                   → "temp" (if not already added)
        setpoint / "sp "       → "setpoint"
        alarm / fault          → "alarm"
        status / "run "        → "status", "run"
        enable                 → "enable"
        damper                 → "damper"
        valve                  → "valve"
        fan                    → "fan"
        pump                   → "pump"
        supply / discharge     → "discharge"
        return / exhaust       → "return"
        outside / oat          → "outside"
```

2.3 Rebuild

Function: `rebuild_tags_for_serials(db, serial_numbers)`

1. Delete all rows from `HAYSTACK_POINT_TAGS` for given serials
2. Read all INPUTS, OUTPUTS, VARIABLES for those serials
3. Call `auto_tag_point()` on every row
4. Return count of points tagged

2.4 Search

Function: `search_points(db, SearchPointsRequest)`

Filters: `device_serials`, `point_types`, `tag_filter`, `label_filter`, `units_filter`. Returns matching `PointTagEntry` rows.

---

3. REST API Routes

File: `api/src/t3_device/haystack_tags_routes.rs` **Registered in:** `api/src/server.rs` via `create_haystack_tags_routes()`

Route	Method	Handler	Description
`/api/haystack/tags`	GET	`list_tags`	List all tags. Query: `?filter=haystack|custom`
`/api/haystack/tags`	POST	`create_tag`	Create custom tag. Body: `{tagName, doc?}`
`/api/haystack/tags/:name`	PUT	`update_tag`	Update doc or deprecated flag. Body: `{doc?, deprecated?}`
`/api/haystack/tags/:name`	DELETE	`delete_tag_handler`	Delete tag (custom only, must be unused)
`/api/haystack/tag-tree`	GET	`get_tag_tree`	Tag hierarchy. Returns `TagTreeNode[]` with children
`/api/haystack/point-tags/read`	POST	`read_point_tags`	Body: `{serialNumbers, pointType?}`
`/api/haystack/point-tags/write`	POST	`write_point_tags`	Body: `BatchPointTagUpdate[]` with `serial_number, point_type, point_index, point_id, add_tags?, remove_tags?, set_tags?`
`/api/haystack/replace-tag`	POST	`replace_tag`	Body: `{oldTag, newTag}`. Updates all point references
`/api/haystack/rebuild`	POST	`rebuild_tags`	Body: `{serialNumbers}`. Re-runs auto-tagging
`/api/haystack/sync`	POST	`sync_official_tags`	Fetches `project-haystack.org/defs.json`, reseeds standard tags

All routes use local SQLite (`local_config_conn`) — never the center DB.

4. Rust Code Structure

```TEXT
api/
  ├── migration/src/
  │   ├── m20260607_add_haystack_v2.rs       ← Creates 3 tables + indexes
  │   └── m20260607_remove_haystack_v1.rs    ← DROP TABLE HAYSTACK_ENTITY
  │
  └── src/
      ├── server.rs                           ← Registers haystack routes (line 217-218)
      ├── entity/t3_device/
      │   └── haystack_entity.rs              ← SeaORM entity (deprecated — v1 table dro
      └── t3_device/
          ├── mod.rs                          ← pub mod declarations
          ├── haystack_tags_service.rs         ← Service: CRUD, tree, auto-tag, rebuild,
          ├── haystack_tags_routes.rs          ← Axum route handlers (10 endpoints)
          └── t3_ffi_sync_service.rs          ← Calls auto_tag_point() during FFI sync (
```

FFI integration (`t3_ffi_sync_service.rs`): After each INPUTS/OUTPUTS/VARIABLES row is inserted or updated, calls `auto_tag_point()` with the point's label, digital_analog, and derived units.

---

5. Frontend

5.1 Store

File: `src/t3-react/features/haystack/store/haystackStore.ts` **Library:** Zustand

State: `tags` , `tagTree` , `pointTags` , `isLoading` , `error` , `selectedTag` Actions: `fetchTags` , `fetchTagTree` , `fetchPointTags` , `createTag` , `updateTag` , `deleteTag` , `replaceTag` , `batchUpdatePointTags` , `setSelectedTag`

5.2 Pages

URL	Component	File
`/t3000/haystack-tags`	`HaystackTagsPage`	`features/haystack/pages/HaystackTagsPage.tsx`
`/t3000/custom-tags`	`CustomTagsPage`	`features/haystack/pages/CustomTagsPage.tsx`

HaystackTagsPage: Standard tags (`category=haystack`). Interactive tree (expand/collapse/filter). "Sync" button → `POST /api/haystack/sync` .

CustomTagsPage: Create/edit/delete custom tags. Inline editing. Delete confirmation. Only custom tags can be deleted (standard tags protected server-side).

5.3 Tags Column (Input/Output/Variable Pages)

Components:

- `TagsColumnCell.tsx` — Renders tags inline in the data grid
- Tag Drawer — Opens on click, allows adding/removing tags per point

Data flow:

1. Page loads → `fetchTagsForDevice(serialNumber)` → `POST /api/haystack/point-tags/read`
2. Tags column renders chip-style inline tags (max 2 visible, `+N` overflow)
3. Click tag cell → drawer opens with full tag list + add/remove controls
4. Save → `POST /api/haystack/point-tags/write`

5.4 Menu & Routes

Menu config: `src/t3-react/config/menuConfig.ts`

- "Haystack Tags" → `#/t3000/haystack-tags` (Alt+Y)
- "Haystack Custom Tags" → `#/t3000/custom-tags`

Routes: `src/t3-react/app/router/routes.ts`

- `/t3000/haystack-tags` → `HaystackTagsPage`
- `custom-tags` nested route → `CustomTagsPage` (in `App.tsx`)

PageHeader: `src/t3-react/layout/PageHeader.tsx`

- `/t3000/haystack-tags` → breadcrumb: "Haystack"
- `/t3000/custom-tags` → breadcrumb: "Custom Tags" under "Haystack"

---

6. API Request/Response Examples

List Tags

```
TEXT

GET /api/haystack/tags?filter=haystack
→ { "tags": [{ tag_name, doc, category, deprecated, source, usage_count, parents }], "to
```

Read Point Tags

```
TEXT

POST /api/haystack/point-tags/read  { "serialNumbers": "237219" }
→ { "entries": [{ serial_number, point_type, point_index, point_id, tag_name }], "total"
```

Write Point Tags

```
TEXT

POST /api/haystack/point-tags/write  [{ serialNumber, pointType, pointIndex, pointId, se
→ { "message": "Point tags updated", "count": 1 }
```

Sync

```
TEXT

POST /api/haystack/sync
→ { "success": true, "message": "Standard tags synced...", "count": 243 }
```

Rebuild

```
TEXT

POST /api/haystack/rebuild  { "serialNumbers": [237219] }
→ { "success": true, "message": "Haystack tags rebuilt", "updated": 1, "pointsTagged": 1
```

---

7. Related

Document	Description
v1-haystack-legacy-single-table.md	Deprecated v1 design
v3-haystack-auto-tagging-mcp.md	Planned v3: rules engine + MCP server